



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 4	Tema: Introducción a Docker
	Semana 13	Tutor: Ing. Victoria Castro

Objetivos de Aprendizaje

Al finalizar la lectura y la práctica de esta guía, serás capaz de:

- Comprender el paradigma de la contenedorización y su importancia en el despliegue de aplicaciones web modernas.
- Diferenciar técnica y conceptualmente una Máquina Virtual (VM) de un Contenedor Docker.

- Dominar el ciclo de vida básico de Docker (Imágenes, Contenedores y Registros).
- Desplegar un servidor web local y entender el mapeo de puertos para el acceso desde el navegador.

El Reto de Nuestro Semestre y Por Qué Surge Docker

A lo largo del semestre, hemos venido construyendo una arquitectura web completa y profesional:

1. Un **Backend** con **Spring Boot 3** (Java).
2. Una **Base de Datos** relacional con **PostgreSQL**.
3. Un **Frontend** interactivo con **React** (JavaScript).

Hasta ahora, para trabajar, cada uno ha tenido que instalar Java 17+, configurar el usuario y puertos de Postgres localmente, e instalar Node.js para React. ¿Qué pasa si el proyecto final se debe evaluar en la computadora del docente o desplegar en la nube?

- El infierno de "en mi máquina sí funciona" aparece debido a sutiles diferencias de versiones de Java, variables de entorno de Postgres mal configuradas o dependencias de Node rotas.

La Solución de la Unidad 4: Docker nos permitirá empaquetar nuestro backend de Spring Boot, nuestra base de datos Postgres y nuestro frontend de React en "cajas" independientes, aisladas y ligeras llamadas contenedores. Al finalizar la unidad, desplegaremos las tres tecnologías simultáneamente con un solo comando, garantizando que funcione idéntico en cualquier computadora del mundo. Esta semana, iniciaremos entendiendo los fundamentos de esta herramienta.

Sección 1: ¿Qué es Docker?

Docker es una plataforma de código abierto que permite automatizar el despliegue de aplicaciones dentro de contenedores.

Concepto Clave: Un contenedor es una unidad estándar de software que empaqueta el código de tu aplicación web (por ejemplo, tu API de Spring Boot) junto con todas sus

dependencias (el entorno de ejecución de Java, librerías, configuraciones) para que se ejecute de forma rápida y aislada en cualquier sistema operativo sin necesidad de instalar nada directamente en la máquina anfitriona.

Sección 2: Arquitectura: Contenedores vs. Máquinas Virtuales

Es común confundir ambos conceptos, pero su arquitectura interna es radicalmente distinta. La clave está en **qué se virtualiza**.

1. **Máquinas Virtuales (VM):** Virtualizan el **hardware**. Cada VM incluye un Sistema Operativo (SO) completo. Si quisieras levantar tu API, tu React y tu Postgres de forma aislada usando VMs, necesitarías 3 sistemas operativos pesados corriendo en paralelo. Son lentas y consumen gigabytes de RAM.
2. **Contenedores:** Virtualizan el **Sistema Operativo**. Comparten el mismo Kernel del sistema anfitrión (*Host*). No necesitan un SO propio, lo que los hace extremadamente ligeros (megabytes), eficientes en el uso de memoria y capaces de iniciar en milisegundos.

Característica	Máquina Virtual (VM)	Contenedor (Docker)
Aislamiento	Completo a nivel de Hardware	A nivel de Procesos (mediante el Kernel)
Tamaño en Disco	Pesado (Gigabytes)	Muy Ligero (Megabytes)
Tiempo de Inicio	Minutos / Segundos largos	Milisegundos / Inmediato
Eficiencia	Menor (Duplica recursos de SO)	Alta (Usa los recursos nativos del host)

Sección 3: El Ecosistema Esencial de Docker

Para entender Docker en la ingeniería de software, debes dominar sus tres pilares fundamentales:

1. **Imagen (Image):** Es una plantilla de **sólo lectura** que contiene las instrucciones para crear un contenedor. *Por ejemplo: una imagen oficial de PostgreSQL lista para usar, o una imagen con Java instalado para correr Spring Boot.* Piensa en ella como la *Clase* en la programación orientada a objetos (POO).
2. **Contenedor (Container):** Es la **instancia ejecutable** y viva de una imagen. Siguiendo la analogía, es el *Objeto* instanciado. Es donde tu servidor o base de datos realmente está corriendo y procesando peticiones.
3. **Docker Hub:** Es el registro oficial en la nube (el "GitHub" de las imágenes). Desde aquí podemos descargar (*pull*) imágenes oficiales de tecnologías listas para usar: postgres, node, openjdk, nginx, etc.

Sección 4: Instalación y Verificación

- **Windows / macOS:** Se instala **Docker Desktop** (gestiona el motor de Docker e incluye una interfaz gráfica). *Nota para Windows: Requiere activar WSL2 (Windows Subsystem for Linux).*
- **Linux:** Se instala directamente el **Docker Engine** nativo a través de la terminal de comandos.

Para comprobar que Docker está correctamente instalado y corriendo en tu sistema, abre tu terminal y ejecuta:

```
docker --version
```

Sección 5: Tu Primer Contenedor: Hola Mundo

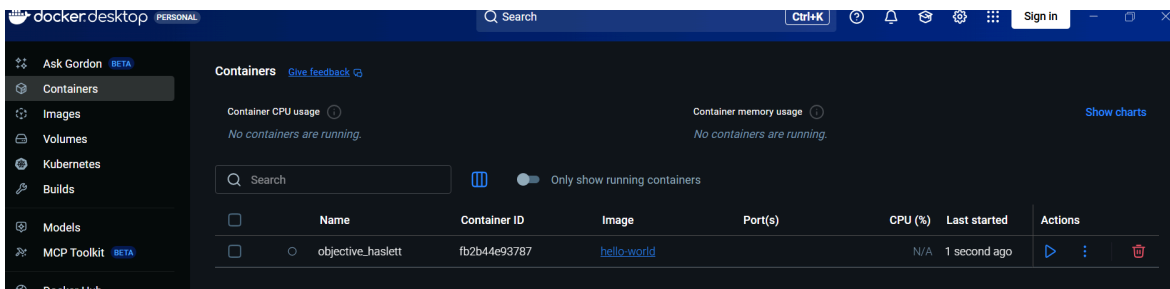
Vamos a comprobar el flujo completo de Docker ejecutando el contenedor de prueba oficial. Ejecuta en tu terminal:

```
docker run hello-world
```

¿Qué ocurrió internamente?

1. El cliente de Docker buscó la imagen hello-world de forma local (en tu disco duro).
2. Al no encontrarla, se conectó automáticamente a Docker Hub.
3. Realizó la descarga (Pull) de la imagen.
4. Instanció y arrancó el contenedor, el cual ejecuta un script que imprime un mensaje de éxito en tu terminal y posteriormente se apaga de forma automática.

NOTA: Si estás trabajando con Windows asegúrate que Docker Desktop esté corriendo primero para ejecutar el comando en la terminal.



```

C:\Users\Victoria>docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:0e760fd4bc48ba8041e7c6db999bb40bfca508b4be580ac75d32c4e29d202ce1
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
  
```

Sección 6: Laboratorio Práctico: Desplegando un Servidor Web (Nginx)

Para simular el despliegue de una aplicación web o estática en producción de manera local, utilizaremos Nginx (uno de los servidores web más rápidos y utilizados del mundo).

Ejecuta el siguiente comando en tu terminal:

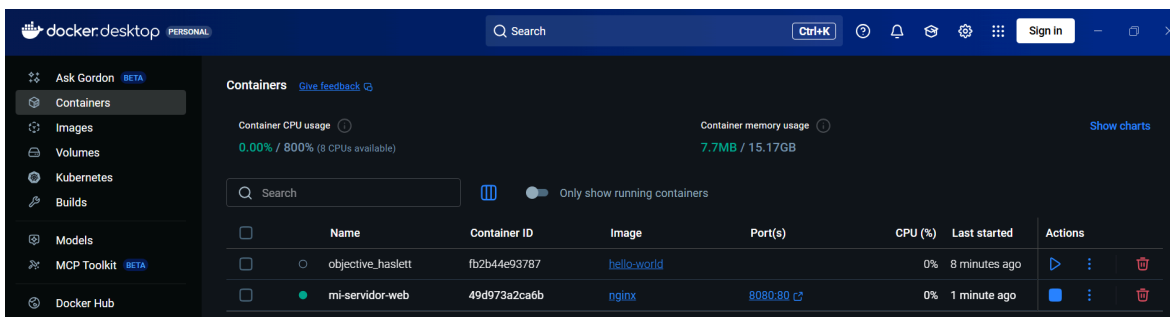
```
docker run -d -p 8080:80 --name mi-servidor-web nginx
```

Desglose técnico del comando:

- `docker run`: Comando para crear e iniciar un contenedor.
- `-d` (Detached): Ejecuta el contenedor en **segundo plano**. Libera tu terminal para que puedas seguir trabajando mientras el servidor web sigue corriendo.
- `-p 8080:80` (Port Mapping): Vincula el puerto **8080 de tu computadora física** (Host) con el puerto **80 interno del contenedor** (donde Nginx escucha por defecto).
- `--name mi-servidor-web`: Asigna un nombre personalizado al contenedor para no tener que usar IDs complejos al detenerlo o borrarlo.
- `nginx`: Nombre de la imagen oficial que Docker buscará y descargará desde Docker Hub.

Verificación: Abre tu navegador web favorito (Chrome, Firefox, Edge) e ingresa a la dirección: `http://localhost:8080`. Verás la página de bienvenida oficial de Nginx corriendo de forma 100% aislada dentro del contenedor.

```
C:\Users\Victoria>docker run -d -p 8080:80 --name mi-servidor-web nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
13fd728be9eb: Pull complete
7f8b1a2b17d8: Pull complete
5431d0092ffd: Pull complete
45381ecb0e2f: Pull complete
b4a248c845e5: Pull complete
5b4d6ff92fc4: Pull complete
830625e1ac85: Pull complete
cc5f57206478: Download complete
5e6b66b5e5f1: Download complete
Digest: sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
Status: Downloaded newer image for nginx:latest
49d973a2ca6b8749fc46dba31fdb2b1c8487f8336ac92e6044dc3143b041157b0
C:\Users\Victoria>
```



The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Ask Gordon, Containers, Images, Volumes, Kubernetes, Builds, Models, MCP Toolkit, and Docker Hub. The main area is titled 'Containers' and shows system metrics: Container CPU usage at 0.00% / 800% (8 CPUs available) and Container memory usage at 7.7MB / 15.17GB. Below these metrics is a table of running containers.

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	objective_haslett	fb2b44e93787	hello-world		0%	8 minutes ago	
☒	mi-servidor-web	49d973a2ca6b	nginx	8080:80	0%	1 minute ago	



The screenshot shows a web browser window with the address bar set to `http://localhost:8080`. The page content is as follows:

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
 To engage with the community please visit community.nginx.org.
 For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

Sección 7: Preguntas de Reflexión (Autoevaluación)

Contesta las siguientes preguntas en el foro semanal en campus virtual:

- Pensando en nuestro proyecto del semestre: ¿Qué ventajas creen que nos dará empaquetar nuestra base de datos PostgreSQL en Docker en lugar de que cada integrante la instale de forma tradicional en su sistema operativo?
- Si en el comando de Nginx cambiamos el parámetro a `-p 3000:80`, ¿en qué dirección URL del navegador tendríamos que buscar nuestra aplicación web?
- ¿Cuál es la diferencia conceptual entre una Imagen de Docker y un Contenedor de Docker?

Bibliografía y Fuentes Oficiales

- [Docker Get Started Guide](#) - Documentación oficial interactiva.
- [Docker Hub Reference](#) - Repositorio para explorar imágenes de tecnologías web (como la de postgres que usaremos pronto).